Guía de Trabajos Prácticos: Programación Orientada a Objetos en PHP

Introducción

Para todos los ejercicios se requiere:

- Uso de **Namespaces** siguiendo el estándar PSR-4.
- Configuración de **Composer** para el Autoloading.
- Tipado estricto en parámetros y retornos (ej: `public function metodo(string $p): void`).

Ejercicio 1: El Sistema de Logística (Herencia e Interfaces)

La empresa "PHP-Express" desea gestionar su flota y costos de envío.

1. **Modelo de Clases**:
   - **Interfaz** `Enviable`: Debe definir el método `calcularCostoEnvio(): float`.
   - **Clase Abstracta** `Vehiculo`: Debe poseer los atributos: `patente`, `marca` y `modelo`. Debe construirse obligatoriamente con estos tres datos.
   - **Clase** `Camion`: Hereda de `Vehiculo` e implementa `Enviable`. Posee el atributo `capacidadCarga` (en kg).
   - **Clase** `Motocicleta`: Hereda de `Vehiculo` e implementa `Enviable`. Posee el atributo `cilindrada` (en CC).
2. **Lógica de Negocio**:
   - En `Camion`, el costo de envío es: **$100 * capacidadCarga**.
   - En `Motocicleta`, el costo de envío es: **$500 (base) + (2 * cilindrada)**.
3. **Implementación**:
   - Cree una clase `GestionLogistica` que permita agregar objetos de tipo `Enviable`.
   - Implemente un método `obtenerCostoTotal(): float` que devuelva la sumatoria de los costos de envío de todos los vehículos registrados.

Ejercicio 2: El Gimnasio de Alto Rendimiento (Traits)

Se desea calcular la proximidad de los socios a las diferentes sedes del gimnasio.

1. **Modelo de Clases**:
   - **Trait** `Geolocalizable`: Debe contener los atributos `latitud` (float) y `longitud` (float), y un método `distanciaA(float $lat, float $long): float` que devuelva la diferencia absoluta de la suma de las coordenadas.
   - **Clase** `Socio`: Posee `nombre`, `dni` y utiliza el Trait `Geolocalizable`.
   - **Clase** `Sede`: Posee `nombreSede` y utiliza el Trait `Geolocalizable`.
2. **Objetivo**:
   - Instanciar un socio y una lista de 3 sedes con diferentes coordenadas.
   - Implementar un programa que recorra las sedes e informe cuál es la más cercana al socio.

Ejercicio 3: Sistema de Notificaciones (Polimorfismo con Interfaces)

Una plataforma debe enviar alertas por diferentes canales según la preferencia del usuario.

1. **Modelo de Clases**:
   - **Interfaz** `Notificador`: Define el método `enviar(string $mensaje): void`.
   - **Clases**: `EmailNotificador`, `SmsNotificador` y `PushNotificador`. Cada una debe imprimir en pantalla un mensaje simulando el envío (ej: "Enviando Email: [mensaje]").
   - **Clase** `Usuario`: Posee `nombre`, `email`, `telefono` y un arreglo donde se almacenan objetos que implementen `Notificador`.
2. **Objetivo**:
   - Implementar en `Usuario` el método `agregarCanal(Notificador $canal)`.
   - Implementar el método `notificarTodo(string $mensaje)` que recorra los canales y ejecute el envío en cada uno.

Ejercicio 4: E-Commerce de Productos Digitales (Clases Abstractas)

Una tienda desea administrar su catálogo de cursos y software.

1. **Modelo de Clases**:
   - **Clase Abstracta** `Producto`: Posee `nombre` y `precioBase`. Define el método abstracto `getPrecioFinal(): float`.
   - **Clase** `Curso`: Posee `cantidadHoras`. El precio final es: **precioBase + 10%**.
   - **Clase** `Software`: Posee `version`. El precio final es: **precioBase + 21% (IVA)**.
2. **Objetivo**:
   - Implementar una clase `Carrito` que almacene objetos de tipo `Producto`.
   - Implementar el método mágico `__toString()` en la clase `Carrito` para que al imprimir el objeto se liste: Nombre del producto, Precio Final y, al finalizar, el Total General de la compra.

Ejercicio 5: Biblioteca Digital UNLP (Repaso General)

Gestión avanzada de recursos de aprendizaje.

1. **Modelo de Clases**:
   - **Interfaz** `Buscable`: Define el método `contienePalabra(string $termino): bool`.
   - **Trait** `Auditable`: Define un atributo `fechaUltimoAcceso` y el método `registrarAcceso(): void` (asigna la fecha actual).
   - **Clase Abstracta** `Recurso`: Posee `id` y `titulo`.
   - **Clase** `LibroPDF`: Hereda de `Recurso`, usa el Trait `Auditable` e implementa `Buscable`. Posee el atributo `texto`.
   - **Clase** `VideoTutorial`: Hereda de `Recurso`, usa el Trait `Auditable` e implementa `Buscable`. Posee el atributo `transcripcion`.
2. **Objetivo**:
   - `contienePalabra` debe devolver `true` si el término existe en el `texto` o en la `transcripcion` respectivamente.

- Cree un programa que instancie varios recursos. Realice una búsqueda de una palabra clave. Si el recurso la contiene, debe imprimir el título y ejecutar `registrarAcceso()`.